7-Day Jupyter Notebook Practice Project

Prepare to independently create, run, explain, debug and document realistic notebook workflows, then take the interview on SME Careers.

How to use this guide

Download `jupyter_7day_practice_pack.zip` and unzip it. Inside you get one folder per day (`day1` to `day7`) with the datasets, plus `answer_keys/answer_checks.md` . **Do not open the answer key until you finish the day.**

Each day follows the same pattern: **Goal, Concepts, Tasks, Questions to answer in Markdown, Deliverable, Done when**.

Golden rules for all seven days:

1. Create **one notebook per day**, named `dayN_topic.ipynb` , saved in the same folder as that day's data.
2. Every notebook starts with a Markdown title cell and an objective.
3. Write a short Markdown cell **before each code block** (what and why) and **after each result** (what it shows).
4. At the end of every day: **Kernel > Restart Kernel and Run All Cells**. If anything fails, fix it before you stop.
5. Never copy code you cannot explain line by line. Interviewers test understanding, not typing speed.

Recommended setup: `pip install jupyterlab pandas numpy matplotlib seaborn scikit-learn openpyxl` , then run `jupyter lab` from the pack folder. Google Colab also works; upload the files for each day.

What each day covers

Day	Topic	Dataset	Main outcome
1	Jupyter fundamentals	`day1_sales.csv`	Clean, documented first notebook
2	Python fundamentals	`day2_machine_log.txt` , `day2_parts_inventory.csv`	Functions, loops, files, exceptions

Day	Topic	Dataset	Main outcome
3	Pandas cleaning and analysis	`day3_production_raw.csv`, `day3_lines_lookup.xlsx`	Messy data to clean CSV plus defect analysis
4	Visualization	`day4_monthly_kpi.csv` (and your Day 3 clean file)	Six or more charts with explanations
5	Debugging and file management	`day5_broken_notebook.ipynb`, `day5_orders.xlsx`	Seven bugs fixed and explained
6	Machine learning basics	`day6_defect_prediction.csv`	Trained and evaluated classifier
7	Mock assessment	`day7_deliveries_raw.csv`, `day7_suppliers.xlsx`	Full end-to-end task, screen-recorded

---

Day 1: Jupyter Notebook fundamentals

Goal: Be fluent with the notebook environment and produce a clean, readable notebook.

Concepts: `.ipynb` files, code cells vs Markdown cells, execution order, execution counters `In [n]`, kernel restart, relative vs absolute paths, exporting.

Dataset: `day1/day1_sales.csv` (150 rows: `date, region, product, units_sold, unit_price`).

Tasks

1. Create `day1_jupyter_basics.ipynb`. Add a Markdown title cell with a `#` heading, a bold objective sentence, and a bulleted list of what you will do.
2. Import `pandas` and `matplotlib.pyplot` in the first code cell.
3. Load the CSV with a **relative path**. Then load it again with an **absolute path** and note which would break if you moved the notebook to another computer.
4. Show `df.head()`, `df.shape`, `df.dtypes`.
5. Create a column `revenue = units_sold * unit_price`.
6. Calculate total revenue, revenue per region, and the best-selling product by units.
7. Make one simple bar chart of revenue per region.
8. Deliberately run cells **out of order** (for example run the revenue cell before the load cell). Observe the error and the execution numbers.
9. Restart the kernel and run all cells.

10. Export to HTML: **File > Save and Export Notebook As > HTML**.

Questions to answer in Markdown cells at the bottom

- What is the difference between a code cell and a Markdown cell?
- What happens to variables in memory when you restart the kernel?
- Why can a notebook look correct but fail when someone else runs it?
- What is the difference between a relative and an absolute path, and which is better for sharing?
- What does the number in `In [7]` tell you?

Deliverable: `day1_jupyter_basics.ipynb` + `.html` export.

Done when: Restart and Run All works with no errors, and every section has a heading.

Day 2: Python fundamentals

Goal: Write and explain core Python without leaning on Pandas.

Concepts: variables, lists, dicts, tuples, sets, conditionals, loops, functions, lambda, list comprehensions, exception handling, reading and writing files.

Datasets: `day2/day2_machine_log.txt` (200 log lines like `2025-03-01 06:00:00 | LINE-A | WARNING | Temperature high 80.2C`) and `day2/day2_parts_inventory.csv`.

Tasks (use plain Python and the `csv` module only)

1. Open the log file and read it line by line into a list. Print the number of lines.
2. For each line, split on `|` and strip whitespace into a dictionary with keys `timestamp, line, level, message`. Store all dictionaries in a list.
3. Write a function `count_by_level(records)` that returns a dict such as `{"INFO": 120, "WARNING": 40, "ERROR": 40}`.
4. Use a **list comprehension** to build a list of all ERROR messages. Use a **set** to get the unique error types.
5. Write a function `errors_per_line(records)` returning which production line has the most errors.
6. Use `try/except` to safely open a file that does not exist and print a friendly message instead of crashing.
7. Read `day2_parts_inventory.csv` with the `csv` module. Write a function that returns every part where `stock < reorder_level`.
8. Write the low-stock parts to a new file `low_stock_parts.csv`.

9. Use `lambda` with `sorted()` to sort parts by `unit_cost` descending and print the top 5.
10. Write `calculate_average(numbers)` that returns `0` for an empty list. Test it with `[]`, `[5]`, and `[78, 85, 92, 67, 88]`.

Questions to answer in Markdown

- When would you use a list, a dict, a tuple, and a set?
- What is the difference between `==` and `is`?
- Why do we use `with open(...)` instead of plain `open(...)`?
- What does a list comprehension do, and when is a normal loop clearer?
- Which exceptions did you meet today, and what caused each one?

Deliverable: `day2_python_basics.ipynb` + `low_stock_parts.csv`.

Done when: Every function has a one-line docstring and at least one test call underneath.

Day 3: Pandas cleaning and analysis (highest priority)

Goal: Turn a messy manufacturing dataset into a clean, analysed one. This is the core of your whole assessment.

Concepts: `read_csv`, `read_excel`, `info`, `describe`, `isnull`, `drop_duplicates`, `fillna`, `astype`, `to_datetime`, `str` methods, `loc` / `iloc`, `groupby`, `merge`, `to_csv`.

Datasets: `day3/day3_production_raw.csv` and `day3/day3_lines_lookup.xlsx` (two sheets: `Lines`, `Products`).

Columns in the raw file: `record_id, date, production_line, shift, product, operator_id, units_planned, units_produced, units_defective, downtime_minutes, temperature_c, supplier`.

The data is **deliberately messy**: duplicates, inconsistent text, placeholder values, missing data, impossible sensor readings and mixed date formats. Find them yourself first.

Part A: Inspect

1. Load the CSV. Show shape, `info()`, `describe()`, and `isnull().sum()`.
2. List every unique value in `production_line` and `shift`. Write down what is wrong.
3. Look at `units_produced`: why is its type `object` instead of a number?
4. Check `temperature_c` with `describe()`. Which values are not physically realistic?

Part B: Clean (explain each decision in Markdown)

5. Count exact duplicate rows, then remove them.
6. Standardise `production_line` to exactly five values: `Line A` to `Line E`. Standardise `shift` to title case.
7. Convert `units_produced` to numeric (`pd.to_numeric(..., errors="coerce")`).
8. Parse `date` into a real datetime. Handle the two formats present.
9. Replace impossible temperatures (the sensor-error value) with `NaN`.
10. Decide how to treat missing values. Suggested approach: fill `downtime_minutes` with the median, fill `temperature_c` with the median per line, and **drop** rows where `units_produced` is missing. Justify your choice.

Part C: Analyse

11. Create `defect_rate = units_defective / units_produced`.
12. Create `efficiency = units_produced / units_planned`.
13. Which production line has the highest **overall** defect rate? (Hint: compute total defective / total produced per line, not the mean of row rates. Explain why that matters.)
14. Compare defect rate by `shift`, by `product`, and by `supplier`.
15. Compute the average downtime per line. Does more downtime go with more defects?
16. Use `df.loc[...]` to list all records with `defect_rate > 0.10`. Then use `iloc` to show the first 5 rows and first 4 columns.

Part D: Merge and export

17. Read the `Lines` sheet with `pd.read_excel(..., sheet_name="Lines")` and merge it onto your cleaned data on `production_line`.
18. Read the `Products` sheet, merge on `product`, and compute `cost_of_defects = units_defective * unit_cost_eur`.
19. Which line has the largest total `cost_of_defects`? Which product is **above its `target_defect_rate`**?
20. Export the cleaned, merged dataset to `output/day3_production_clean.csv` (create the folder in code with `os.makedirs`).

Questions to answer in Markdown

- What is the difference between `loc` and `iloc`?
- When is dropping missing values acceptable, and when is it risky?
- Why compute defect rate as total divided by total instead of averaging row-level rates?

- What does `how="left"` do in `merge`?
- How did you confirm that your cleaning worked? (Show a check cell for each fix.)

Deliverable: `day3_pandas_cleaning.ipynb` + `output/day3_production_clean.csv`.

Done when: Your final row count, line spellings, and null counts match the self-check section in `answer_keys/answer_checks.md`.

Day 4: Data visualization

Goal: Produce meaningful charts and say what each one tells you.

Concepts: Matplotlib figure and axes, titles and labels, Seaborn themes, choosing the right chart.

Datasets: `day4/day4_monthly_kpi.csv` (`month, production_line, units_output, units_defective, oee, energy_kwh, downtime_hours`, 12 months x 5 lines) and your own `day3_production_clean.csv`.

Tasks

For every chart: set a figure size, title, axis labels, and legend if needed, then write **two sentences** underneath saying what you see and what you would recommend.

1. **Line chart:** total `units_output` per month (group by month first).
2. **Multi-line chart:** monthly defect rate per production line (compute `units_defective / units_output`). Which line is worst, and is there a seasonal pattern around July and August?
3. **Bar chart:** average `oee` per line, sorted from highest to lowest.
4. **Histogram:** distribution of `temperature_c` from your Day 3 clean data (try 20 bins).
5. **Box plot (Seaborn):** `defect_rate` by `shift` using Day 3 data.
6. **Scatter plot:** `downtime_minutes` vs `defect_rate`, coloured by `production_line`.
7. **Correlation heatmap:** numeric columns of the Day 3 clean data with `sns.heatmap(df.corr(numeric_only=True), annot=True)`.
8. Save at least three of the charts as PNG with `plt.savefig("output/chart_name.png", dpi=150, bbox_inches="tight")` **before** `plt.show()`.
9. Create one **2x2 subplot figure** combining four of the charts.

Questions to answer in Markdown

- When do you choose a line chart over a bar chart?

- What does a box plot show that a bar chart of averages hides?
- What does a correlation of 0.6 mean, and why is correlation not causation?
- Which chart would you show to a manager, and why?

Deliverable: `day4_visualization.ipynb` + a PNG chart folder.

Done when: A stranger can read your notebook and understand the story without reading any code.

Day 5: Debugging and file management

Goal: Find, explain and verify fixes for realistic errors. Interviewers love this.

Concepts: reading tracebacks (read from the **bottom** up), `ModuleNotFoundError`, `FileNotFoundError`, `NameError`, `KeyError`, `TypeError`, `ValueError`, paths, execution order.

Datasets: `day5/day5_broken_notebook.ipynb` and `day5/day5_orders.xlsx`.

Part A: Fix the broken notebook

Open `day5_broken_notebook.ipynb`. It contains **seven deliberate bugs**. Restart and run all, then fix them one at a time. After each fix, add a Markdown cell using this template:

> **Bug N:** the error message **Cause:** why it happened **Fix:** what I changed **Verification:** how I proved it is fixed

You are done only when **Restart and Run All** finishes cleanly and the file `output/clean_orders.csv` exists. Then compare your list with the answer key.

Part B: File management drills

In a new notebook, `day5_file_management.ipynb`:

1. Print the current working directory with `os.getcwd()` and list the files with `os.listdir()`.
2. Use `pathlib.Path` to build a path to a data file without hard-coding slashes.
3. Create an `output/` folder if it does not exist.
4. Read the Excel file with the correct header row, strip the whitespace from every column name, and convert `unit_price` to float.
5. Export one CSV and one Excel file (`df.to_excel("output/orders_clean.xlsx", index=False)`).

6. Write a small `requirements.txt` listing the libraries your notebook needs and explain how someone else would install them.

Part C: Deliberately break things

Intentionally create each of these errors in a scratch notebook, read the traceback, then fix it: `ModuleNotFoundError` (import a misspelled library), `FileNotFoundError`, `NameError`, `KeyError`, `TypeError`, `ValueError` (for example `int("abc")`).

Questions to answer in Markdown

- How do you read a traceback?
- What is the first thing you check when you get a `KeyError` in Pandas?
- Why can an out-of-order cell cause a `NameError` that disappears after you rerun?
- What is the difference between `pip install` in a terminal and `%pip install` in a notebook?

Deliverable: fixed notebook, `day5_file_management.ipynb`, `output/` folder.

Done when: You can explain each bug in under 30 seconds without looking at your notes.

Day 6: Machine learning basics

Goal: Run a clean, simple ML workflow and explain the results honestly.

Concepts: features and target, train/test split, classification, accuracy, precision, recall, F1-score, confusion matrix, class imbalance.

Dataset: `day6/day6_defect_prediction.csv` (1,200 rows). Target: `defective_batch` (1 = defective). Features: `line, shift, temperature_c, humidity_pct, machine_age_years, operator_experience_months, cycle_time_sec, supplier_quality_score`.

Tasks

1. Load the data. Check shape, dtypes, missing values, and the **balance of the target** (`value_counts(normalize=True)`).
2. Handle missing values in `humidity_pct` and `supplier_quality_score` (median imputation is fine). Explain your choice.
3. Convert `line` and `shift` into numeric columns with `pd.get_dummies(..., drop_first=True)`.
4. Separate `X` (features) from `y` (target).

5. Split with `train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)`.
6. Train a **baseline** that always predicts "not defective". Calculate its accuracy. Notice how high it is.
7. Train `LogisticRegression(max_iter=1000)` and then `RandomForestClassifier(random_state=42)`.
8. For each model print accuracy, precision, recall, F1-score (`classification_report`) and the confusion matrix.
9. Plot the random forest `feature_importances_` as a horizontal bar chart.
10. Re-train with `class_weight="balanced"`. What changes in recall for the defective class?

Questions to answer in Markdown

- Why do we split data into training and test sets?
- Why is accuracy misleading on this dataset? Which metric would a factory care about most, and why?
- What do precision and recall mean in plain words for defective batches?
- What is overfitting, and how could you notice it?
- Which features seem to matter most, and does that make physical sense?

Deliverable: `day6_ml_basics.ipynb`.

Done when: You can explain the confusion matrix to a non-technical manager in three sentences.

Day 7: Full mock assessment (record your screen)

Goal: Do a complete, unseen task end to end, exactly like the real assessment. This scenario is **new**, so you cannot rely on memory from earlier days.

Scenario: You are a data analyst for a manufacturer. The purchasing manager wants to know which suppliers are reliable, which are slow, and which deliver poor-quality parts.

Datasets: `day7/day7_deliveries_raw.csv` and `day7/day7_suppliers.xlsx` (sheets `Suppliers`, `Parts`).

Rules

- Set a timer for **90 minutes**.

- **Record your screen** with a tool such as OBS Studio, Loom, or the built-in recorder of your operating system. Talk through your decisions aloud.
- Open a new notebook called `day7_supplier_analysis.ipynb`. Do not look at earlier notebooks until you are stuck for more than five minutes.

Required workflow and structure (use these Markdown headings)

1. **Objective**
2. **Data loading** (CSV plus both Excel sheets)
3. **Data inspection** (shape, types, missing values, duplicates, odd values)
4. **Data cleaning** (document every decision)
5. **Feature engineering** (create `lead_time_days`, `reject_rate`, `fill_rate`)
6. **Analysis**
7. **Visualization** (at least three charts)
8. **Conclusion and recommendations**
9. **How to reproduce** (Python version, libraries, file locations, run order)

Analysis questions

1. How many rows are in the raw file, and how many remain after cleaning?
2. What problems did you find in `supplier`, `quantity_ordered`, `status`, and the dates?
3. Calculate `lead_time_days = delivery_date - order_date` and the average per supplier.
4. Compare each supplier's average lead time against its `contract_lead_time_days`. Who breaks the contract most?
5. Calculate `reject_rate = rejected_units / quantity_received` per supplier. Who is worst?
6. Calculate `fill_rate = quantity_received / quantity_ordered`. Which part category is delivered most completely?
7. Merge in `Parts` and compute the total value of rejected units in euros.
8. What share of deliveries are Delayed, Partial or Cancelled for each supplier?
9. Rank suppliers using a simple scorecard (lead time, reject rate, fill rate) and recommend which supplier to review first.
10. Export `output/day7_deliveries_clean.csv` and `output/supplier_scorecard.csv`.

Final checklist before you stop recording

- [ ] Kernel > Restart and Run All completes with no errors.
- [ ] Each section has a Markdown explanation and a conclusion.

- [ ] Charts have titles, labels and a sentence of interpretation.
- [ ] Output files exist in `output/`.
- [ ] A "How to reproduce" section tells another person exactly how to run it.
- [ ] Notebook exported to HTML.

Deliverable: notebook, HTML export, two output CSVs, screen recording.

Interview preparation: SME Careers

I do not have the details of how the SME Careers assessment is run, so treat the format below as a safe assumption and check the instructions on the platform carefully before you begin. Based on the job description you shared, expect a **recorded practical task in Jupyter Notebook**: you create, run and explain a realistic workflow, then submit the notebook and recording.

Before the interview

- Read the platform instructions twice. Note the **time limit, allowed tools, required file formats and recording rules**.
- Test your microphone, screen recorder, internet, and that your Jupyter environment starts in under a minute.
- Prepare a clean working folder: `data/`, `output/`, and the notebook. Close unrelated tabs and notifications.
- Practise the Day 7 workflow once more on a fresh day with a different public dataset.

Strategy during the task

1. Spend the first minutes **reading the whole task** and writing a plan in Markdown.
2. Work in small steps: load, inspect, clean, analyse, visualise, conclude.
3. Talk out loud: say **what** you are doing and **why**.
4. When you hit an error, read the traceback calmly, explain the cause, then fix it. A well-explained bug fix scores higher than perfect code.
5. Keep the notebook tidy: no unused experiments, no cell with 200 lines.
6. Finish with **Restart and Run All**, then export and save everything.

Questions to rehearse aloud (60 seconds each)

Question	Key points to cover
What is Jupyter Notebook?	Interactive document that mixes code, output and Markdown text
Code cell vs Markdown cell?	Code is executed by the kernel; Markdown renders formatted documentation
What happens when you restart the kernel?	All variables are cleared from memory; the file on disk is unchanged
How do you load an Excel file?	`pd.read_excel(path, sheet_name=...)`, needs `openpyxl` installed
How do you handle missing values?	Inspect with `isnull().sum()`, then drop, fill (mean, median, mode) or flag, and justify
`loc` vs `iloc`?	`loc` uses labels and conditions; `iloc` uses integer positions
How do you find duplicates?	`df.duplicated().sum()`, then `drop_duplicates()`
How do you visualize a dataset?	Pick a chart for the question, label it, interpret it
How do you debug a notebook?	Read traceback bottom-up, check variable and column names, check execution order, restart and run all
How do you ensure reproducibility?	Relative paths, fixed random seeds, `requirements.txt`, ordered cells, Restart and Run All, documented steps
How do you document a task for someone else?	Clear headings, assumptions, decisions, a "How to reproduce" section, screen recording

Common mistakes to avoid

- Hard-coded absolute paths that only exist on your computer.
- Cells that only work because of a variable created in a deleted cell.
- Charts without titles or interpretation.
- Dropping missing rows without explaining why.
- Submitting without running **Restart and Run All**.

- Silent cleaning: always show a before and after check.

Suggested pacing

If you have less than seven days, compress as follows. **Two days:** Day 3 and Day 7. **Four days:** Days 1+2, Day 3, Days 4+5, Day 7. Skip Day 6 only if the job description does not mention machine learning.

Self-assessment: are you ready?

You are ready when you can answer yes to all of these:

- I can create a notebook, structure it with Markdown, and run it top to bottom without errors.
- I can load CSV and Excel files, including a specific sheet and a header offset.
- I can clean text, dates, numbers, duplicates and missing values and explain each choice.
- I can group, merge, calculate rates and answer a business question with data.
- I can make and interpret at least six chart types.
- I can read an error message and fix it calmly while explaining aloud.
- I can train and evaluate a basic classifier and explain precision and recall.
- I can finish a 90-minute end-to-end task and leave a reproducible result.

If you want, send me your Day 3 or Day 7 notebook and I will review it like a technical evaluator, point out what is missing, and run a spoken-style mock interview with you one question at a time.